



PlayToGet

Project Documentation and Deployment Guide

Document scope

This PDF describes the PlayToGet application, its main modules, architecture, local Docker setup, production deployment workflow, maintenance tasks, and operational recommendations. Sensitive credentials are intentionally represented as placeholders.

Field	Value
Application	PlayToGet sports social network and community portal
Framework	Laravel 13, PHP 8.3
Primary database	MySQL 8.0
Admin panel path	/cp
Prepared	19 June 2026
Repository	https://github.com/janickiy/playtoget3.git

Prepared for project handover, deployment, and ongoing maintenance.

Table of Contents

1. Project Overview
2. Core Features
3. Technology Stack
4. Architecture and Code Organization
5. Project Structure
6. Local Docker Deployment
7. Production Deployment
8. Environment Configuration
9. Database, Storage, and Media
10. Admin Panel
11. Operations and Maintenance
12. Security Checklist
13. Testing and Troubleshooting
14. Useful Commands

1. Project Overview

PlayToGet is a sports-focused social network and community platform. It helps users discover sports communities, join teams and groups, create events, exchange messages, publish media, and find sports-related places such as playgrounds, shops, and fitness clubs.

The application combines a public-facing portal, authenticated user areas, media galleries, user-to-user interaction tools, and an administrative panel for moderation and content management.

- Users can maintain profiles, upload avatars and cover images, manage privacy settings, and track online status.
- Communities are represented as teams and groups with members, administrators, invitations, privacy rules, posts, photos, videos, and events.
- Events support membership, invitations, albums, video albums, comments, likes, sharing, and calendar visibility.
- Sport blocks represent playgrounds, shops, and fitness clubs with detailed pages, media galleries, and owner-managed content.
- The administration panel provides structured CRUD tools, moderation statuses, logs, settings, menu management, feedback handling, and cache controls.

Audience

This document is intended for developers, administrators, DevOps engineers, and marketplace reviewers who need to understand, install, deploy, or maintain the project.

2. Core Features

Module	Description
Home and news feed	Main feed, user posts, attached photos, comments, likes, and sharing.
Profiles	Personal information, contacts, privacy settings, avatars, covers, albums, messages, blacklist, and online status.
Friends	Friend requests, incoming and outgoing invitations, friend lists, and interaction shortcuts.
Teams and groups	Community pages, membership roles, invitations, private or closed access, administrators, blacklist, photos, videos, and events.
Events	Event listings, date filtering, calendar integration, members, invitations, albums, video albums, comments, likes, and sharing.
Media galleries	Photo albums, video albums, modal viewers, comments, likes, share actions, and attachment support.
Sport blocks	Playgrounds, shops, and fitness club directories with owner editing, media sliders, statuses, and recommendations.
Search	Global search by teams, groups, events, and sport blocks with grouped results.
Feedback	Validated feedback form, reCAPTCHA v3 protection, admin processing statuses, and email notifications.
Admin panel	Dashboard, users, admins, communities, events, sport blocks, announcements, settings, menus, content pages, feedback, and logs.

3. Technology Stack

Layer	Technology
Backend	PHP 8.3, Laravel 13, MVC controllers, Form Requests, Services, Repositories, DTOs, Enums.
Database	MySQL 8.0 as the primary database. PostgreSQL is present as an optional secondary Docker service.
Cache and queues	Laravel cache, database queue driver by default, Redis and Memcached services available in Docker.
Frontend assets	Blade templates, JavaScript, jQuery, Vite, Tailwind tooling, updated media libraries.
Admin UI	AdminLTE-style panel with Yajra DataTables packages.
Authentication	Local authentication and Laravel Socialite providers: Google, Facebook, X, and LinkedIn OpenID.
Security add-ons	reCAPTCHA v3 package for feedback forms, Laravel validation, bcrypt-only password checks.
Media processing	GD extension, storage services, configured media paths in config/media.php.
Containerization	Docker Compose with PHP-FPM, nginx, MySQL, PostgreSQL, Redis, Memcached, and phpMyAdmin.

4. Architecture and Code Organization

The project follows Laravel conventions but also separates business operations into dedicated layers. Controllers should remain thin and delegate validation, data mapping, persistence, media handling, and notification behavior to specialized classes.

- **Controllers** handle HTTP requests and route users to views or JSON responses.
- **Form Requests** validate input for profile updates, content forms, admin CRUD screens, feedback, and media forms.
- **DTO classes** carry validated data from requests into repositories and services.
- **Repositories** encapsulate database operations for users, profiles, communities, events, sport blocks, albums, messages, settings, and content.
- **Services** handle media storage, image transformations, cascade deletion, notification emails, news feed formatting, profile updates, and online status logic.
- **Enums** define domain statuses such as user, community, event, sport block, and feedback states, including display labels and AdminLTE color classes.

Important controllers

The main frontend controllers include ProfileController, FriendsController, GroupsController, TeamsController, EventsController, PhotoalbumsController, VideoalbumsController, PlaygroundsController, ShopsController, FitnessController, FeedbackController, AnnouncementsController, ContentController, CalendarController, NewsController, HomeController, and AjaxController.

The admin panel is served from routes/cp.php and includes DashboardController, AdminController, UsersController, CommunitiesController, EventsController, SportBlocksController, AnnouncementsController, FeedbackController, LogsController, SettingsController, ContentController, MenuController, CacheController, and authentication controllers.

5. Project Structure

Path	Purpose
app/Http/Controllers/Front	Frontend controllers for public pages and authenticated user flows.
app/Http/Controllers/Admin	Admin panel controllers under the /cp prefix.

Path	Purpose
app/Http/Requests	Laravel Form Request validation classes.
app/Repositories	Database access and domain persistence operations.
app/DTO	Data transfer objects used between requests, services, and repositories.
app/Services	Domain services for media, notifications, profile updates, settings, and cleanup.
app/Models	Eloquent models for users, communities, events, media, feedback, logs, settings, and related tables.
resources/views/front	Frontend Blade templates.
resources/views/admin	Admin panel Blade templates.
routes/web.php	Frontend and AJAX routes.
routes/cp.php	Admin panel routes with /cp prefix.
config/media.php	Single source of truth for media directory paths.
docker	Docker configuration for PHP, nginx, MySQL data, PostgreSQL data, and tooling.

6. Local Docker Deployment

The recommended local setup uses Docker Compose. The application container runs PHP-FPM, nginx exposes the web server on port 8080, and MySQL runs as the primary database service.

Prerequisites

- Docker Desktop or Docker Engine with Docker Compose.
- Git.
- Enough disk space for vendor dependencies, node modules, database volumes, and uploaded media.

Step-by-step setup

```
git clone https://github.com/janickiy/playtoget3.git playtoget
cd playtoget
git checkout en
cp .env.example .env
```

Edit `.env` and make sure the local Docker database values match `docker-compose.yml`:

```
APP_NAME=PlayToGet
APP_ENV=local
APP_DEBUG=true
APP_URL=http://localhost:8080
APP_LOCALE=en
APP_FALLBACK_LOCALE=en

DB_CONNECTION=mysql
DB_HOST=mysql_db
DB_PORT=3306
DB_DATABASE=laravel_db
DB_USERNAME=root
DB_PASSWORD=root_password

CACHE_STORE=database
QUEUE_CONNECTION=database
FILESYSTEM_DISK=public
MAIL_MAILER=log
```

Start containers and install the application dependencies:

```
docker compose up -d --build
docker compose exec app composer install
docker compose exec app php artisan key:generate
docker compose exec app php artisan storage:link
docker compose exec app php artisan migrate --seed
docker compose exec app npm install
docker compose exec app npm run build
```

Open the local application at <http://localhost:8080>. The admin panel is available at <http://localhost:8080/cp>. phpMyAdmin is available on port 8089 when the tools profile is enabled.

```
docker compose --profile tools up -d phpmyadmin
# phpMyAdmin: http://localhost:8089
```

Local demo credentials

The default local admin credentials are admin / 1234567. Change all default passwords before any public deployment.

7. Production Deployment

Production deployment should use a clean checkout, a production `.env` file, optimized Composer dependencies, compiled frontend assets, writable storage directories, and a configured web server pointing to the Laravel public directory.

Example production target

Item	Example
Domain	https://playtoget.janickiy.com
Application directory	<code>/var/html/playtoget</code>
Branch	<code>en</code>
Database	MySQL database playtoget with a dedicated playtoget user
Public web root	<code>/var/html/playtoget/public</code>

Server deployment workflow

```
ssh root@YOUR_SERVER_IP
cd /var/html

git clone https://github.com/janickiy/playtoget3.git playtoget
cd /var/html/playtoget
git checkout en
cp .env.example .env
```

Configure `.env` for production. Do not store real secrets in documentation, screenshots, or Git history.

```

APP_NAME=PlayToGet
APP_ENV=production
APP_DEBUG=false
APP_URL=https://playtoget.janickiy.com
APP_LOCALE=en
APP_FALLBACK_LOCALE=en

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=playtoget
DB_USERNAME=playtoget
DB_PASSWORD=<DB_PASSWORD>

CACHE_STORE=database
QUEUE_CONNECTION=database
FILESYSTEM_DISK=public
MAIL_MAILER=smtp

```

Install and optimize:

```

composer install --no-dev --optimize-autoloader
php artisan key:generate --force
php artisan storage:link
php artisan migrate --force
npm install
npm run build
php artisan optimize:clear
php artisan config:cache
php artisan route:cache
php artisan view:cache

```

Set ownership and permissions according to the web server user used on the VPS:

```

chown -R www-data:www-data /var/html/playtoget
find /var/html/playtoget/storage -type d -exec chmod 775 {} \;
find /var/html/playtoget/bootstrap/cache -type d -exec chmod 775 {} \;

```

nginx virtual host example

```

server {
    listen 80;
    server_name playtoget.janickiy.com;
    root /var/html/playtoget/public;
    index index.php index.html;

    client_max_body_size 64M;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        include fastcgi_params;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock;
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
        fastcgi_param DOCUMENT_ROOT $realpath_root;
    }

    location ~ /\.(!well-known).* {
        deny all;
    }
}

```

Enable HTTPS with Certbot or your preferred TLS automation tool. After issuing certificates, redirect HTTP traffic to HTTPS and verify APP_URL uses https.

8. Environment Configuration

The .env file controls application mode, URLs, database connections, cache, queue, mail, social login providers, reCAPTCHA, and frontend naming. Keep .env outside version control.

Variable group	Important variables
Application	APP_NAME, APP_ENV, APP_KEY, APP_DEBUG, APP_URL, APP_LOCALE, APP_FALLBACK_LOCALE
Database	DB_CONNECTION, DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, DB_PASSWORD
Cache and queues	CACHE_STORE, QUEUE_CONNECTION, REDIS_HOST, MEMCACHED_HOST
Mail	MAIL_MAILER, MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_FROM_ADDRESS, MAIL_FROM_NAME
Social login	GOOGLE_CLIENT_ID, FACEBOOK_CLIENT_ID, X_CLIENT_ID, LINKEDIN_CLIENT_ID and matching secrets/redirect URIs
reCAPTCHA v3	RECAPTCHA_V3_SITEKEY, RECAPTCHA_V3_SECRET, RECAPTCHA_V3_LOCALE

OAuth redirect URLs

Provider callback URLs follow /auth/{provider}/callback, for example <https://playtoget.janickiy.com/auth/google/callback>. Configure each provider dashboard with the same URL as the production .env file.

9. Database, Storage, and Media

The project stores structured data in MySQL and media files under Laravel storage/public. The public storage symlink must exist so uploaded files are available through the web server.

Database lifecycle

- Use php artisan migrate --force for production schema changes.
- Use php artisan migrate:fresh --seed only for clean demo installations because it destroys existing data.
- Use mysqldump for backups before deploying migrations or replacing production data.
- One migration file should represent one table or one focused schema change.

```
# Backup
mysqldump -u playtoget -p playtoget > playtoget_backup.sql

# Restore
mysql -u playtoget -p playtoget < playtoget_backup.sql

# Apply migrations
php artisan migrate --force
```

Media path configuration

Media directories are centralized in config/media.php. Services should use this configuration instead of hard-coded paths. Current directory keys include user_avatar, user_cover, profile_tmp_avatar, profile_tmp_cover, event_cover, sport_block_avatar, gallery, gallery_user, gallery_user_attach, attachment_comment, and attachment_message.

Cascade deletion services are responsible for removing database records and associated media files when events, sport blocks, teams, or groups are deleted.

10. Admin Panel

The administration panel is mounted under /cp and uses role or permission middleware for protected sections. It provides moderation and content management tools for the application.

Admin section	Capabilities
Dashboard	Main admin landing page and cache reset entry point.
Users	View, edit, block, unblock, soft delete, bulk actions, profile status colorization.
Admins	Administrative user CRUD.
Communities	Teams and groups moderation, status changes, recommended flag, owner and media overview.
Events	Event moderation, statuses, hidden or blocked state, editing and deletion.
Sport blocks	Playgrounds, shops, and fitness administration with statuses and recommendation flag.
Announcements	Full CRUD for announcements shown on the frontend.
Feedback	Status management, answers, and email notifications.
Content and menus	Static pages, slugs, published state, and menu management.
Settings	Application settings with 5-minute cache lifetime and automatic invalidation.
Logs	Read-only listing of records from the logs table.

Cache behavior

Settings and menus should be cached for five minutes and invalidated automatically when changed in the admin panel. The manual cache reset button is available for administrators.

11. Operations and Maintenance

Recurring operational tasks

- Run regular database backups and keep at least one off-server copy.
- Monitor storage disk usage because media uploads can grow quickly.
- Review application logs after each deployment.
- Clear and rebuild caches after changing environment variables, routes, translations, or configuration.
- Keep Composer and npm dependencies updated carefully, with browser checks for media libraries and jQuery-related changes.
- Verify queue workers if the queue driver is changed from database/log behavior to a real background worker setup.

Common maintenance commands

```
php artisan optimize:clear
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan migrate --force
php artisan queue:restart
php artisan storage:link
```

12. Security Checklist

- Never commit .env, database dumps with real data, OAuth secrets, SSH keys, or production credentials.
- Set APP_DEBUG=false in production.
- Use HTTPS for all production traffic.
- Change default admin credentials immediately after installation.

- Use strong database passwords and a dedicated database user with the minimum required privileges.
- Configure social login redirect URLs exactly and restrict provider applications to trusted domains.
- Keep reCAPTCHA v3 keys private and environment-specific.
- Validate uploads by MIME type and extension; keep upload directories outside executable PHP paths.
- Run backups before migrations and before importing database dumps.
- Review roles and admin permissions after creating or importing users.

13. Testing and Troubleshooting

Recommended verification after deployment

- Open the homepage and verify the top navigation, search form, sidebars, and recommended blocks.
- Log in as a normal user and verify profile editing, privacy tabs, dialogs, photo upload, video albums, and friend requests.
- Open teams, groups, events, playgrounds, shops, and fitness pages and confirm hidden or blocked statuses are not visible on the frontend.
- Open /cp, log in as an administrator, and verify users, settings, menus, logs, feedback, announcements, events, communities, and sport blocks.
- Submit feedback and confirm validation, reCAPTCHA, database storage, and email notification behavior.
- Upload a photo and confirm storage/public links work correctly.

Troubleshooting quick table

Problem	Checks
Blank page or 500 error	Check storage/logs/laravel.log, APP_DEBUG in local only, PHP version, Composer install, and cached config.
Database connection error	Verify DB_HOST, DB_DATABASE, DB_USERNAME, DB_PASSWORD, network access, and Docker service names.
Uploaded images are missing	Run php artisan storage:link, verify FILESYSTEM_DISK, permissions, and config/media.php directories.
Admin route not found	Confirm routes/cp.php is loaded, route cache is rebuilt, and the URL starts with /cp.
Social login fails	Check provider client IDs, secrets, callback URLs, HTTPS, and APP_URL.
Feedback form fails	Check RECAPTCHA_V3_SITEKEY, RECAPTCHA_V3_SECRET, mail settings, and validation errors.

14. Useful Commands

```

# Local Docker lifecycle
docker compose up -d --build
docker compose ps
docker compose logs -f app
docker compose exec app bash
docker compose down

# Laravel application
php artisan about
php artisan route:list
php artisan migrate:status
php artisan optimize:clear
php artisan storage:link

# Dependencies
composer install
composer update --dry-run
npm install
npm run build

# Testing
php artisan test
./vendor/bin/phpunit

# Production deployment refresh
git pull origin en
composer install --no-dev --optimize-autoloader
npm install
npm run build
php artisan migrate --force
php artisan optimize:clear
php artisan config:cache
php artisan route:cache
php artisan view:cache

```

Appendix: Main Frontend URLs

URL	Purpose
/	Home page and main search entry.
/profile/{id}	User profile page.
/profile/edit	Profile settings and privacy tabs.
/friends	Friends and friend requests.
/photoalbums	Current user photo albums.
/videoalbums	Current user video albums.
/teams	Team directory and team search.
/groups	Group directory and group search.
/events	Event directory and date filtering.
/calendar	Calendar view with event dates.
/playgrounds	Sports playground directory.
/shops	Sports shop directory.
/fitness	Fitness club directory.
/announcements	Announcement listing.
/feedback	Feedback form.
/page/{slug}	Published content page by slug.

URL	Purpose
/cp	Admin panel.

Final note

For a production handover, pair this document with a current database backup, a sanitized .env template, and a short release checklist for the exact hosting environment.